PRODUCT MEMO

The Company That Knows Itself

The system of record for how a company actually operates

EXECUTIVE SUMMARY

Organizations run on institutional knowledge: policies, procedures, workflows, decisions, commitments, and operational experience accumulated across documents, teams, and systems. As organizations grow, that knowledge fragments across departments, evolves over time, and increasingly diverges from operational reality. Existing approaches either retrieve documents (RAG) or generate answers (LLMs), but neither preserves how organizational knowledge relates, changes, conflicts, or performs in practice.

This product builds a continuously evolving organizational knowledge graph that transforms organizational artifacts into structured, traceable knowledge. It captures official knowledge, operational reality, cross-functional consistency, and temporal evolution in a single representation. The result is a living knowledge layer that can be queried by humans, consumed by AI agents, and continuously validated against real-world outcomes.

A working end-to-end prototype has been implemented and demonstrated on a synthetic organizational corpus. The system extracts typed claims, constructs structured relationships, builds a queryable graph, and supports evolution tracing, contradiction detection, workflow reconstruction, and organizational drift analysis. The current prototype contains more than 150 nodes and 170 typed relationships generated across three organizational document sets.

The opportunity is driven by the rapid adoption of AI agents inside enterprises. As agents become responsible for support, operations, engineering, and decision-making workflows, organizational knowledge becomes critical infrastructure rather than documentation. Every agent requires reliable ground truth. Today, that infrastructure does not exist.

The technical thesis has been implemented and demonstrated. The primary wedge is B2B software companies — organizations where policy-to-engineering drift, cross-functional inconsistency, and the gap between official knowledge and operational reality are structural and recurring. The AI agent deployment wave makes this infrastructure critical now: every internal agent requires reliable organizational ground truth that currently doesn't exist.

We are now in active customer discovery with B2B software companies and are seeking two to three pilot partners to validate the system on production organizational data–companies where policy-to-engineering drift, cross-functional inconsistency, or the gap between documentation and operational reality is already a lived problem.

1. THE PROBLEM

Company knowledge is structurally broken. AI makes it worse.

Every company runs on institutional knowledge — policies, procedures, workflows, decisions. That knowledge is scattered across Confluence, Slack, email threads, support tickets, ADRs, postmortems. Humans navigate it through years of context and asking colleagues. AI agents have no such fallback.

Two specific failures compound this:

- **Engineering vs CS.** Engineering ships a change. CS doesn't know. Support agents spend four days troubleshooting something intentionally deprecated.
- **Sales vs Policy.** Sales commits to a feature on a renewal call. The RFC governing that feature is unresolved. No one connects the two until the contract is signed.
- **Policy vs Reality.** A policy exists but the procedure that implements it was informally replaced six months ago. Every new employee learns the wrong process. Similarly, A policy was superseded six months ago, but the older version remains the one most employees find and follow.

These failures appear different on the surface, but they share the same root cause: organizational knowledge fragments across teams, evolves over time, and gradually diverges from operational reality.

The current technical landscape makes this worse, not better. RAG retrieves relevant documents but does not preserve the organizational relationships between them. LLMs synthesize answers by collapsing evidence into a generated response, often losing provenance, contradictions, uncertainty, and temporal evolution. Neither approach can reliably determine when two documents conflict, when a policy has been superseded, when different teams are operating from different versions of the same information, or when operational reality diverges from official knowledge.

2. THE INSIGHT

There is a missing structural layer between documents and answers.

Every company accumulates organizational knowledge, but existing systems reduce that knowledge to disconnected documents, embeddings, and generated answers. What is missing is a living representation of how a company knows, decides, operates, and changes over time.

The insight is not 'use a graph.' The insight is that institutional knowledge has a specific internal structure that neither vector search nor language model inference preserves:

- Policies govern procedures. Procedures implement workflows. Decisions create policies. Exceptions violate policies. These are organizational relationships, not semantic similarity.
- Knowledge evolves over time. A policy from 2021 was superseded by an ADR in 2023. That evolution is the institutional memory. Flattening it into embeddings destroys it.
- Organizational knowledge is distributed across teams. Engineering, Support, Sales, Product, and Operations each maintain their own view of the company. When those views drift apart, commitments conflict with decisions, procedures conflict with policies, and teams operate from different versions of reality. These inconsistencies must be surfaced and reconciled, not buried inside disconnected documents.
- Official knowledge and operational reality are different systems. Policies describe how work should happen. Incidents, support tickets, and employee behavior reveal how work actually happens. This is a separate layer of signals that presses against official knowledge and must be compared to it, not merged with it.

The common failure is not missing information. It is the loss of organizational context.

The correct representation is a continuously evolving organizational knowledge graph.

The graph operates across two layers:

Layer 1 — Organizational Knowledge

Policies, procedures, workflows, decisions, commitments, and the relationships between them. This layer captures how the company understands its own operations, how knowledge is shared across teams, and how that understanding evolves over time.

Layer 2 — Operational Reality

Incidents, observations, support activity, customer interactions, and other real-world signals. This layer captures how work is actually performed and what outcomes occur in practice.
The graph captures four dimensions of organizational knowledge:

- Structure: how policies, procedures, workflows, decisions, and commitments relate to one another.
- Evolution: how organizational knowledge changes over time.
- Cross-functional consistency: whether different teams operate from the same understanding.
- Operational reality: whether real-world outcomes align with official knowledge.

The result is a living organizational knowledge layer that preserves institutional memory, makes organizational drift visible, enables knowledge transfer across employees, and provides continuously evolving ground truth for both humans and AI agents.

3. WHY NOW

Five forces converged. The window just opened.

This product could not have been built five years ago. It becomes possible and necessary because five independent shifts happened at the same time.

1. **Foundation models unlocked structured organizational knowledge.**
   Recent foundation models made it practical to automatically extract structured claims, relationships, and organizational concepts from unstructured company data. This makes a continuously evolving organizational knowledge graph feasible for the first time.
2. **AI agents turned knowledge quality into a critical dependency.**
   Humans compensate for incomplete documentation through context, experience, and conversations. AI agents cannot. As agents become responsible for support, operations, coding, and decision-making workflows, the quality of the underlying knowledge layer becomes a system-level constraint.
3. **Enterprise agent adoption is accelerating.**
   Gartner projects that 40% of enterprise applications will include task-specific AI agents by the end of 2026, up from less than 5% in 2025. Every one of these agents requires reliable organizational knowledge to operate correctly.
4. **The infrastructure stack finally exists.**
   Graph databases, vector retrieval systems, GraphRAG techniques, agent frameworks, and foundation models have matured simultaneously. The technical components required to build and maintain a continuously evolving organizational knowledge graph are now available.
5. **The category is beginning to emerge.**
   Independent theses such as Company Brain and AI Operating System describe different manifestations of the same underlying gap. One focuses on making organizational knowledge

executable by agents. The other focuses on creating a closed feedback loop between company knowledge and operational reality. Both require a structured knowledge layer that preserves relationships, history, consistency, and context across the organization.

Organizations have both the capability to build structured organizational knowledge systems and a pressing need to do so. The same forces making AI agents possible are exposing the limitations of existing knowledge infrastructure.

4. PRODUCT

A living organizational knowledge graph.
Organizations run on policies, procedures, workflows, decisions, commitments, and operational signals spread across documents, teams, and systems.

Policies exist in one place. Decisions that changed those policies exist somewhere else. Engineering, Support, Sales, and Operations often maintain different views of the same process. Customer commitments are made in meetings. Operational reality shows up in tickets, incidents, and support conversations.

Most companies do not have a single, reliable representation of how they operate. The knowledge that governs the company is real, but fragmented across documents, teams, and time. As organizations evolve, knowledge changes, teams drift apart, and reality gradually diverges from official understanding.

This product turns that fragmented knowledge into a continuously evolving organizational knowledge layer. It captures official knowledge, operational reality, and the relationships between them, making it possible to understand how knowledge is structured, how it evolves, whether teams operate from the same understanding, and where reality diverges from official expectations.

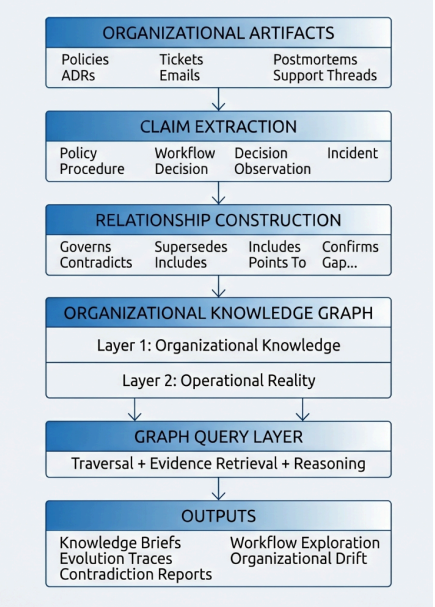
What it enables

- Unified organizational knowledge across documents, teams, and time.
- Evolution tracking showing what changed, when it changed, and what replaced it.
- Cross-functional consistency detection across engineering, support, sales, operations, and policy.
- Organizational drift detection between official knowledge and operational reality.
- Structured workflow and procedure discovery.
- Fully traceable answers with source-level provenance.
- Structured knowledge packets that preserve relationships, evolution, authority, and provenance.

Reasoning Modes
The graph supports four primary reasoning modes.
Knowledge Briefs retrieve the current organizational position on a topic with full provenance.
Evolution Traces reconstruct how policies, workflows, decisions, and commitments changed over time.
Contradiction Analysis identifies conflicts within official knowledge, across teams, or between official knowledge and operational reality.
Procedure Exploration reconstructs workflows, dependencies, governing policies, and execution paths across the organization.

All outputs are generated from graph evidence and remain traceable to their original sources.

5. ARCHITECTURE



The system stores organizational knowledge as claims rather than documents. A claim is an atomic, falsifiable, and traceable statement that can be verified, contradicted, superseded, or connected to other claims while remaining linked to its original source. Claims are extracted from organizational artifacts and classified into concepts such as policies, procedures, workflows, decisions, commitments, observations, and incidents.

Knowledge is organized into two layers. Layer 1 captures organizational knowledge: the policies, procedures, workflows, decisions, and commitments that define and govern how the company operates. Layer 2 captures operational reality: incidents, observations, support activity, customer interactions, and other signals that describe how work is actually performed. These layers remain separate and are continuously compared rather than merged.

Information enters the system from organizational sources such as policies, ADRs, tickets, postmortems, support conversations, and operational records. Documents are first transformed into structured representations from which claims are extracted. These claims become graph nodes, and relationships are then constructed between them based on organizational context. The resulting graph preserves governance relationships, workflow structure, policy evolution, cross-functional dependencies, contradictions, and links between official knowledge and operational outcomes while maintaining full provenance throughout graph construction.

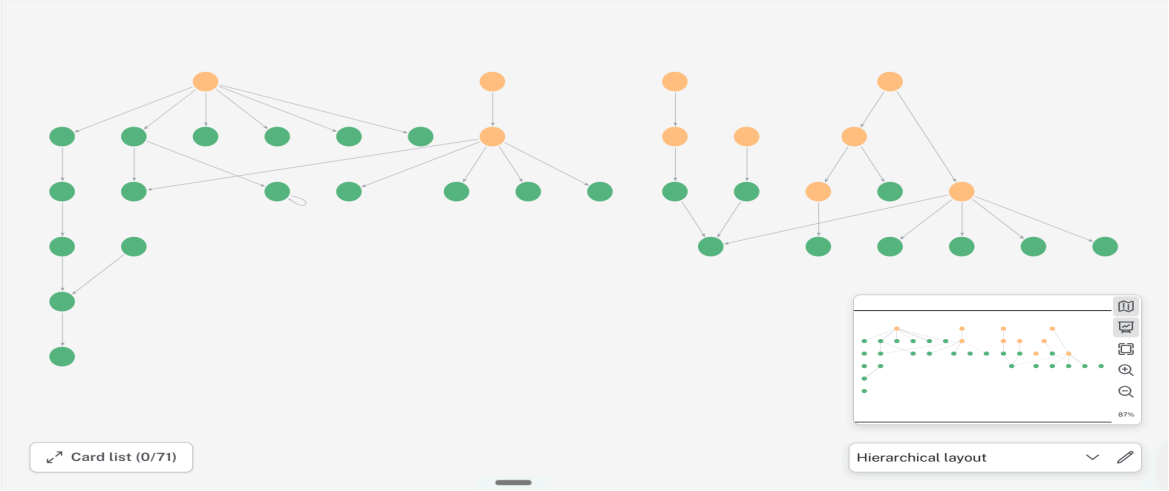
The two-layer knowledge graph is queried directly rather than summarized through probabilistic generation. Queries retrieve graph evidence, relationships, and supporting context while preserving the original organizational structure. A core design constraint is non-compression retrieval: every statement in the output remains traceable to graph nodes, and every graph node remains traceable to its source. The system therefore presents structured evidence rather than generated conclusions, making the output suitable for both human decision-making and AI agent consumption.

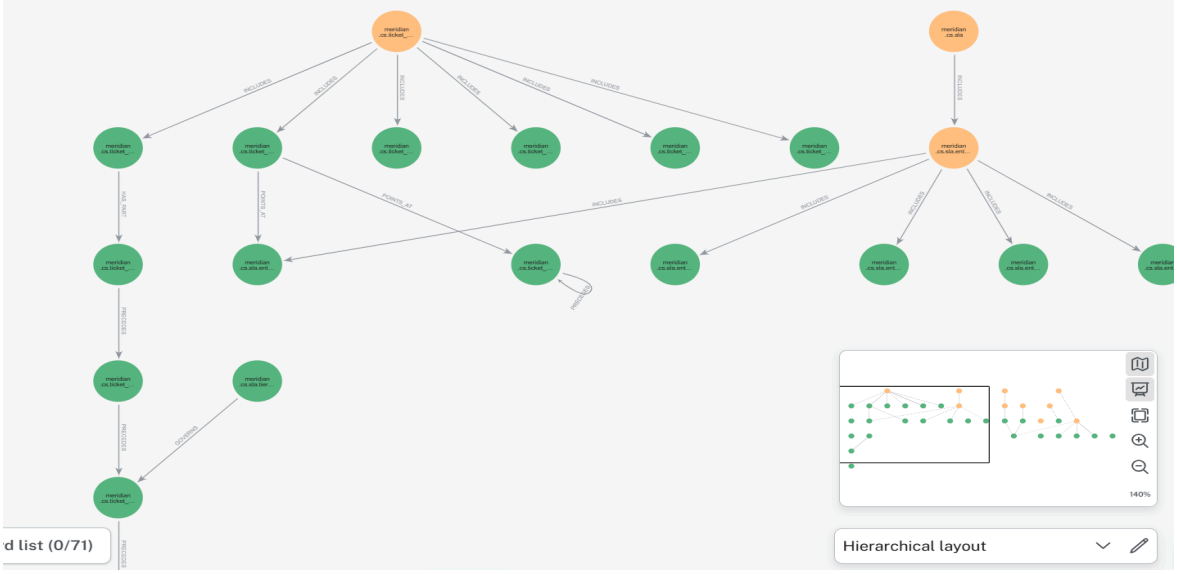
The architecture is designed to extend beyond static graph construction. As new organizational information enters the system through operational sources, it is continuously compared against existing graph knowledge to identify contradictions, confirmations, exceptions, policy drift, emerging patterns, and changes in organizational behavior. These signals can be incorporated back into the organizational representation, allowing the graph to evolve alongside the company rather than remain a snapshot of historical knowledge. Over time, recurring operational signals form a higher-level representation of organizational reality, preserving not only what the organization knows, but also how it learns, adapts, and resolves conflicts. This feedback loop transforms the graph from a knowledge repository into a continuously evolving organizational learning system.

6. PROTOTYPE

The current prototype implements the complete architecture described above. Using three organizational document sets(of Meridian Company-synthetically generated), the system automatically extracts claims, constructs typed relationships, and builds a queryable organizational knowledge graph. The current graph contains more than 150 nodes and 170 relationships spanning policies, procedures, workflows, decisions, commitments, observations, and incidents. All graph outputs remain traceable to source evidence and support the reasoning modes described above.

The following examples demonstrate the graph structure and the reasoning capabilities operating on top of it.





01 What is Meridian's PostgreSQL support policy and how has it changed?

EVOLUTION

1. 2023-03-15: Meridian offers two deployment modes: Meridian Cloud (managed SaaS) and Meridian Self-Hosted. (source: DOC1, DOC1_p002)
2. 2023-03-15: Self-hosted installations are provided as-is under the terms of the Meridian Community License. (source: DOC1, DOC1_p003)
3. 2023-03-15: Meridian Support does not provide dedicated engineering time for self-hosted issues. (source: DOC1, DOC1_p003)
4. 2023-03-15: Users running self-hosted deployments are expected to maintain their own infrastructure, including database upgrades, network configuration, and storage provisioning. (source: DOC1, DOC1_p003)
5. 2023-10-20: Meridian's current support policy (CS Policy v2.1) lists PostgreSQL 12 as the minimum validated version. (source: DOC3, DOC3_p001)
6. 2023-10-20: The informal guidance some CS agents have provided about PG14 being safe in practice now becomes formally true. (source: DOC3, DOC3_p005)
7. 2023-10-20: Any customers who acted on informal guidance regarding PG14 and hit issues will now have a legitimate support claim. (source: DOC3, DOC3_p005)

Official position as of 2023-10-20: Meridian's current support policy lists PostgreSQL 14 as formally supported, and any issues arising from previous informal guidance on PG14 are now legitimate support claims. (source: DOC3, DOC3_p005)
UNRESOLVED: The original policy document (DOC1) has not been updated to reflect the change in minimum validated PostgreSQL version from 12 to 14. Readers of DOC1 currently receive outdated information. (source: DOC3, DOC3_p001)
Sources consulted: DOC1 (2023-03-15) — Initial support policy details, DOC3 (2023-10-20) — Updated support policy and changes.
(*Model: Amazon Nova Pro)* .

02 Generate a current-state knowledge brief on Meridian's PostgreSQL support policy for a new CS agent joining today.

KNOWLEDGE BRIEF

The timeline of Meridian's support policy concerning PostgreSQL shows the following evolution:
On 2023-03-15, Meridian offers two deployment modes: Meridian Cloud (managed SaaS) and Meridian Self-Hosted (source: DOC1, DOC1_p002, 2023-03-15). Self-hosted installations are provided as-is under the terms of the Meridian Community License (source: DOC1, DOC1_p003, 2023-03-15). Meridian Support does not provide dedicated engineering time for self-hosted issues (source: DOC1, DOC1_p003, 2023-03-15). Users running self-hosted deployments are expected to maintain their own infrastructure, including database upgrades, network configuration, and storage provisioning (source: DOC1, DOC1_p003, 2023-03-15). These policies remain active as components of Meridian's support framework.

On 2023-10-20, Meridian's current support policy (CS Policy v2.1) lists PostgreSQL 12 as the minimum validated version (source: DOC3, DOC3_p001, 2023-10-20). The informal guidance some CS agents have provided about PG14 being safe in practice now becomes formally true (source: DOC3, DOC3_p005, 2023-10-20). Any customers who acted on informal guidance regarding PG14 and hit issues will now have a legitimate support claim (source: DOC3, DOC3_p005, 2023-10-20).
Current official position as of 2023-10-20: Meridian's current support policy (CS Policy v2.1) lists PostgreSQL 12 as the minimum validated version (source: DOC3, DOC3_p001).
Current official position as of 2023-10-20: The informal guidance some CS agents have provided about PG14 being safe in practice now becomes formally true (source: DOC3, DOC3_p005).
Current official position as of 2023-10-20: Any customers who acted on informal guidance regarding PG14 and hit issues will now have a legitimate support claim (source: DOC3, DOC3_p005).
No Layer 2 signals (CONTRADICTS, CONFIRMS, GAP) were found in the provided packet.
No unresolved conflicts, unaddressed gaps, or pending actions were identified in the provided packet.

Sources consulted: Doc1_customer support policy (2023-03-15) — details Meridian's deployment modes, self-hosted license terms, engineering time allocation, and infrastructure responsibility; Doc3_ADR (2023-10-20) — formalizes PostgreSQL 14 guidance, specifies minimum validated PostgreSQL version, clarifies PostgreSQL 14 support claims, and provides inferred categorizations for support policy, customer support, and Meridian.

03 What is the escalation process when a self-hosted support ticket cannot be resolved?

PROCEDURE

The escalation procedure for self-hosted support tickets is a process that CS agents must follow when a self-hosted support ticket is received (source: DOC1, DOC1_p005, 2023-03-15).
The steps in this process are:
1. CS agents must verify if the customer's deployment falls within supported configurations (as defined in Section 2) (source: DOC1, DOC1_p005, 2023-03-15). If the deployment is not in scope, CS agents must send the community redirect template and close the ticket (source: DOC1, DOC1_p005, 2023-03-15).
2. If the customer's deployment is in scope, CS agents must create a Zendesk ticket and assign it to the Tier-1 CS queue (source: DOC1, DOC1_p005, 2023-03-15).
3. If Tier-1 CS agents cannot resolve a ticket within 24 hours, they must escalate it to Tier-2 Senior CS (source: DOC1, DOC1_p005, 2023-03-15).
4. If Tier-2 Senior CS agents require engineering involvement, they must file an internal engineering escalation ticket (source: DOC1, DOC1_p005, 2023-03-15).
5. CS agents must document the resolution of the ticket (source: DOC1, DOC1_p005, 2023-03-15).
6. CS agents must update the internal knowledge base entry for the issue type (source: DOC1, DOC1_p005, 2023-03-15).
Sources consulted: Doc1_customer support policy (2023-03-15) — details customer support policy including escalation procedure.

04 What is the Enterprise SLA and is it being met?

CONTRADICTION

For Enterprise tier accounts, the official policy states a 48-hour initial response Service Level Agreement (SLA) applies to tickets in supported categories only (source: Doc1_customer support policy, DOC1_p006, 2023-03-15). This 48-hour first-response SLA for Enterprise tier customers is also stated as official policy (source: Doc2_supportThread, DOC2_p003, 2023-09-18).
Regarding whether this Enterprise SLA is being met, real-world signals report contradictions.
SIGNAL CONFLICT [signal vs official]:
Official position: For Enterprise tier accounts, a 48-hour initial response Service Level Agreement (SLA) applies to tickets in supported categories only (source: Doc1_customer support policy, DOC1_p006, 2023-03-15)
Real-world signal: Enterprise support tickets are taking significantly longer than the 48-hour SLA.
Reported by: COMMUNITY in DOC2 (source: Doc2_supportThread, DOC2_p001, 2023-09-18)
Status: live (unresolved in available documents). 4. Has any engineering decision created a conflict with CS commitments or procedures? - SIGNAL CONFLICT [signal vs official]:
Official position: The current CS escalation procedure (Section 3 of CS Policy v2.1) requires engineering acknowledgment within 1 business day for escalated tickets (source: Doc3_ADR, DOC3_p004, 2023-10-20).
Real-world signal: Given current engineering team capacity constraints from the 3.6.0 release cycle, the engineering acknowledgment SLA of 1 business day cannot be met for general escalations (source: Doc3_ADR, DOC3_p004, 2023-10-20).
Reported by: AUTHOR in Doc3_ADR on 2023-10-20.
Status: live (unresolved in available documents)
Sources consulted: Doc3_ADR (2023-10-20) — relates to the engineering acknowledgment SLA for escalated tickets and CS escalation procedure.

7. Three Dimensions of Organizational Knowledge

ORGANIZATIONAL KNOWLEDGE	CONSISTENCY & EVOLUTION	OPERATIONAL REALITY
The official knowledge layer. Policies, procedures, workflows, decisions, and commitments represented as structured, versioned knowledge. Queryable by humans and AI systems. Captures how the organization defines and governs its operations. **Status: built.**	The reconciliation layer. Detects contradictions, traces knowledge evolution, surfaces cross-functional inconsistencies, and preserves relationships across documents, teams, and time. This is the layer that transforms isolated information into organizational understanding. **Status: built.**	The reality layer. Operational signals such as incidents, support activity, tickets, conversations, and observations are continuously compared against official knowledge. Organizational drift is surfaced before it becomes customer impact. **Status: architecture built. Live connectors are Stage 3.**

These layers are not independent systems. Official knowledge without consistency and evolution becomes unreliable. Operational signals without a reference point cannot be interpreted. The organizational knowledge graph connects all three, preserving how knowledge is defined, how it changes, and how it performs in practice.

8. PROTOTYPE STATUS

End-to-end pipeline operational on a synthetic organizational corpus.

The core architecture has been implemented end-to-end. The current prototype ingests organizational documents, extracts structured claims, constructs typed relationships, builds a queryable knowledge graph, and supports graph-based reasoning with full provenance.

The prototype operates on a synthetic but realistic organizational corpus (Meridian, a B2B developer observability company) intentionally designed to contain policy evolution, cross-functional inconsistencies, operational deviations, and organizational drift. The current graph contains more than 150 nodes and 170 typed relationships generated across three organizational document sets.

Current Scope

The primary scope of the product is not the architecture but the dataset and scale.
The current prototype operates on three synthetic documents rather than production organizational data. While the core graph construction and reasoning pipeline is functional, several scale-oriented components have not yet been activated because they are unnecessary at the current corpus size. These include:

- Automated document parsing for production sources such as Confluence, Slack, GitHub, and internal knowledge systems.
- Automated paragraph segmentation and document normalization from raw enterprise formats.
- Entity resolution across large document collections.
- Intra-document and cross-document relationship extraction at corpus scale.
- Continuous monitoring and graph updates from live organizational systems.

Path to Production

Stage 2 focuses on production document ingestion and automated graph construction across heterogeneous enterprise data sources.

Stage 3 introduces live organizational monitoring, continuous graph updates, operational signal ingestion, and real-time organizational drift detection between official knowledge and operational reality.

Beyond graph construction, the long-term direction of the system is continuous organizational learning. As operational signals enter the graph, contradictions, confirmations, exceptions, and resolution decisions can be incorporated into the organizational representation over time. This allows the graph to accumulate not only organizational knowledge, but also organizational memory. Every policy change, contradiction, exception, operational signal, and resolution outcome becomes part of a continuously evolving representation of how a specific company operates. Over time, the graph captures recurring patterns, resolution history, and organizational adaptation, making the knowledge layer increasingly valuable as the organization evolves.

9. MARKET AND COMPETITION

Enterprise knowledge infrastructure is becoming a critical dependency for AI adoption. Gartner projects that 40% of enterprise applications will include task-specific AI agents by the end of 2026, up from less than 5% in 2025. As AI systems move from answering questions to performing operational work, the reliability of the underlying organizational knowledge becomes increasingly important.

Existing solutions approach parts of this problem from different directions. Knowledge platforms such as Glean, Guru, Notion AI, and Confluence AI improve retrieval and discovery across organizational content, while emerging agent-memory systems such as Cognee and GraphRAG-based architectures provide structured context for AI systems. Both categories address information access, but neither preserves organizational structure, evolution, cross-functional consistency, and operational reality in a single representation.

The opportunity is to become the system of record for how a company actually operates. By extracting policies, procedures, workflows, decisions, commitments, incidents, and operational signals into a continuously evolving graph, the system makes it possible to answer questions that organizations struggle with today: What is the current policy? What changed? Which team is operating differently? Which customer commitments conflict with engineering reality? Where has operational behavior diverged from the official process?

The demand is driven by operational failures rather than knowledge management itself. A Customer Success manager spends hours reconciling support escalations because engineering constraints conflict with customer-facing commitments and no system surfaces the contradiction. A compliance or operations lead manually traces policies, decisions, and exceptions across documents to determine what is currently authoritative and whether operational processes remain aligned. These failures appear different on the surface but share the same root cause: critical organizational knowledge is distributed across documents, teams, and operational systems without a mechanism to continuously connect, compare, and update it.

10. QUESTIONS UNDER VALIDATION

The technical thesis has been implemented and demonstrated on a synthetic organizational corpus. The primary questions now are market-facing and deployment-oriented.

How much of the organizational knowledge representation is universal, and how much must be adapted for specific domains? While the underlying architecture is domain-agnostic, different industries may require different ontologies, relationship types, and operational signals. The key question is whether a common organizational knowledge layer can serve as the foundation across domains.

We are seeking two to three pilot partners — preferably AI-native or enterprise software companies already deploying internal agents — to validate the system on production organizational data and stress-test the knowledge representation against real organizational complexity. Accelerator support is sought specifically for customer introductions, pilot access, and the feedback loop that comes from deploying in a real organizational environment.